

IES NERVIÓN

Departamento de Informática

PROYECTO FINAL

Programación

Ciclo Formativo: 1º Desarrollo de Aplicaciones Multiplataforma
(DAM)

Módulo: Programación

Modalidad: Individual

Límite: 31 de mayo

Curso: 2025-2026

Índice

1. Introducción y contexto	2
2. Elección de temática	2
3. Requisitos técnicos obligatorios	3
3.1. Estructura y organización del proyecto	3
3.2. Programación Orientada a Objetos (POO)	3
3.3. Herencia y Polimorfismo	4
3.4. Clases Abstractas e Interfaces	4
3.5. Colecciones y Genéricos	4
3.6. Gestión de Excepciones	4
3.7. Entrada/Salida de Ficheros (Persistencia)	5
3.8. Streams y Expresiones Lambda	5
3.9. Interfaz de Usuario por Consola	5
4. Documentación obligatoria	6
4.1. Javadoc	6
4.2. Diagrama UML de Clases	6
4.3. Memoria Técnica	6
5. Apartado de ampliación — Subir nota	7
5.1. Persistencia con Base de Datos — JDBC (<i>hasta +1,5 puntos</i>)	7
5.2. Interfaz Gráfica con JavaFX (<i>hasta +1,5 puntos</i>)	7
5.3. Patrones de Diseño (<i>hasta +1 punto</i>)	8
5.4. Serialización de Objetos (<i>hasta +0,5 puntos</i>)	8
5.5. Hilos y Concurrencia (<i>hasta +1 punto</i>)	8
5.6. Tests Unitarios con JUnit 5 (<i>hasta +1 punto</i>)	8
6. Criterios de evaluación y rúbrica	9
7. Penalizaciones	10
8. Forma de entrega	11
9. Sprints voluntarios	11
10. Consejos finales	12

1. Introducción y contexto

A lo largo del curso has adquirido los fundamentos de la programación orientada a objetos en Java: diseño de clases, herencia, polimorfismo, uso de colecciones, manejo de excepciones, persistencia en ficheros y muchos conceptos más. Ha llegado el momento de demostrar que eres capaz de integrar todos esos conocimientos en un proyecto real, completo y profesional.

Este proyecto es **individual**. Deberás elegir una temática de entre las propuestas (o proponer la tuya con aprobación previa del profesor), diseñar la arquitectura de la aplicación y desarrollarla **completamente desde cero**, sin reutilizar proyectos anteriores.

El objetivo no es solo que el programa funcione, sino que esté **bien diseñado, organizado, documentado y estructurado** como lo haría un desarrollador profesional.

2. Elección de temática

Debes elegir **una** de las siguientes temáticas. Si tienes una idea propia diferente, consúltala con el profesor antes del **primer viernes tras la publicación del enunciado** para su aprobación.

#	Temática	Descripción breve
1	Sistema de gestión de una biblioteca	Gestión de libros, socios, préstamos y devoluciones con control de plazos y multas
2	Sistema de gestión de un hospital	Gestión de pacientes, médicos, especialidades y citas médicas
3	Tienda / comercio electrónico	Catálogo de productos, clientes, carrito, pedidos y control de stock
4	Sistema de gestión de un gimnasio	Socios, actividades, monitores, cuotas mensuales e inscripciones
5	Sistema de gestión de un instituto	Alumnos, profesores, módulos, evaluaciones y boletín de notas
6	Agencia de viajes	Paquetes turísticos, clientes, reservas, destinos y guías turísticos
7	Gestión de una flota de vehículos	Vehículos de distintos tipos, conductores, rutas y mantenimientos
8	Hotel / sistema de reservas	Habitaciones por categorías, clientes, reservas, check-in y check-out
9	Academia de idiomas	Cursos por niveles, alumnos, profesores, matrículas y seguimiento del progreso
10	Juego de rol por consola (RPG)	Personajes con clases y habilidades, combate por turnos, inventario, mazmorras y progresión
11	Videoclub / plataforma de contenidos	Catálogo de películas y series, usuarios, alquileres/suscripciones y valoraciones

#	Temática	Descripción breve
12	Restaurante	Carta con platos y categorías, mesas, pedidos, cuentas y gestión de personal

Aviso: Dos alumnos no pueden elegir la misma temática exacta. Piensa algo parecido y comunica tu elección al profesor **antes del tercer día hábil** tras la publicación.

3. Requisitos técnicos obligatorios

Estos requisitos son de **cumplimiento obligatorio**. Su ausencia implica una penalización directa en la nota según la rúbrica. Todos los conceptos aquí presentes han sido trabajados en clase.

3.1. Estructura y organización del proyecto

- El proyecto debe organizarse en **paquetes Java** con una estructura lógica y coherente. Ejemplo orientativo (adáptalo a tu diseño):

```
src/
|-- modelo/           <- Clases de dominio (entidades)
|-- servicio/         <- Logica de negocio
|-- repositorio/      <- Acceso y persistencia de datos
|-- ui/               <- Interfaz de usuario (menus de consola)
|-- excepciones/      <- Excepciones personalizadas
--- Main.java         <- Punto de entrada
```

- Los nombres de clases, métodos y variables deben seguir las convenciones estándar de Java (camelCase, PascalCase).
- El código debe ser **limpio y legible**: sin código muerto, sin variables sin usar, sin duplicidades evidentes.

3.2. Programación Orientada a Objetos (POO)

Deberás modelar correctamente el dominio de tu aplicación mediante clases bien diseñadas:

- Mínimo 8 clases de dominio** (entidades propias de la temática elegida).
- Aplicación correcta de **encapsulamiento**: todos los atributos **private**, acceso mediante *getters* y *setters* justificados.
- Al menos un ejemplo de **constructor por parámetros**, **constructor de copia** y métodos `toString()`, `equals()` y `hashCode()` sobrescritos de forma coherente.

3.3. Herencia y Polimorfismo

- La jerarquía de clases debe tener **al menos dos niveles de herencia** con sentido semántico real (no forzado).
- Demostrar el uso de **polimorfismo**: referencias a la clase padre o interfaz que en tiempo de ejecución apuntan a objetos de subclases distintas.
- Uso de `@Override` en todos los métodos sobrescritos.

Ejemplo orientativo (biblioteca): `Material` → `Libro`, `Revista`, `DVD`

3.4. Clases Abstractas e Interfaces

- Al menos **una clase abstracta** con métodos abstractos y métodos concretos, justificada por el diseño.
- Al menos **dos interfaces** propias (no de la API estándar) que definan contratos relacionados con la lógica de la aplicación.
 - Ejemplo: `Prestable`, `Buscable`, `Notificable`, `Exportable`, etc.
- Usar interfaces como tipos en listas o parámetros de métodos (no solo como “etiqueta”).

3.5. Colecciones y Genéricos

- Uso de al menos **tres tipos distintos de colecciones** del framework de colecciones de Java, elegidas por adecuación al problema: `ArrayList`, `LinkedList`, `HashMap`, `TreeMap`, `HashSet`, `TreeSet`, `LinkedHashMap`...
- Justificar en la memoria por qué se eligió cada colección concreta.
- Implementar al menos **una clase o método genérico propio** (`<T>`) con sentido real en el diseño.

3.6. Gestión de Excepciones

- Crear al menos **tres excepciones personalizadas** (`extends Exception` o `extends RuntimeException`) propias de la lógica del dominio.
 - Ejemplo (hospital): `PacienteNoEncontradoException`, `CitaDuplicadaException`, `AgendaLlenaException`
- Uso correcto de bloques `try-catch-finally`.
- No usar excepciones para controlar flujo normal del programa.
- Logging básico de errores: mensajes claros y útiles al usuario; nunca un `printStackTrace()` desnudo en producción.

3.7. Entrada/Salida de Ficheros (Persistencia)

La aplicación **debe persistir todos los datos** entre ejecuciones mediante ficheros de texto:

- Usar ficheros **CSV o TXT estructurado** para almacenar y cargar las entidades principales.
- Al arrancar la aplicación, debe **cargar automáticamente** los datos desde fichero.
- Al cerrar la aplicación (opción de menú “Salir”), debe **guardar automáticamente** todos los datos.
- Gestionar correctamente el caso en que el fichero no exista en el primer arranque.
- Usar **BufferedReader / BufferedWriter** (o **Files + Paths** de NIO).
- Los ficheros deben ser legibles por un humano (no binarios).

3.8. Streams y Expresiones Lambda

Demostrar el uso fluido de la **Stream API** en al menos **cinco operaciones distintas** con sentido real. Este es el apartado de ampliación, así que busca documentación e infórmate. Su relación con las expresiones lambda es estrecha, ya que los streams usan lambdas para definir operaciones como filtrar o transformar datos de manera concisa sin modificar la fuente original:

- **filter()** — filtrado por condición
- **map()** — transformación de elementos
- **sorted()** — ordenación con **Comparator** via lambda
- **collect()** — recolección en listas o mapas
- **count()** / **min()** / **max()** / **reduce()** — operaciones de reducción
- Opcional: **groupingBy()**, **partitioningBy()**

No se aceptan usos de Streams triviales o sin propósito. Cada uso debe resolver una consulta real del sistema.

3.9. Interfaz de Usuario por Consola

- La aplicación debe tener un **menú principal** claro y navegable, con submenús según las funcionalidades.
- Las opciones deben incluir como mínimo: **alta, baja, modificación, consulta/-listado** de las entidades principales (CRUD completo).
- Validación de todas las entradas del usuario: entradas numéricas fuera de rango, cadenas vacías, formatos incorrectos, etc.
- El programa no debe terminar abruptamente bajo ninguna entrada del usuario.
- Mensajes de confirmación y error claros y en español.

4. Documentación obligatoria

4.1. Javadoc

Todas las clases, interfaces, atributos y métodos públicos deben estar documentados con Javadoc:

```
1  /**
2   * Representa un socio de la biblioteca.
3   *
4   * @author Nombre Apellido
5   * @version 1.0
6   */
7  public class Socio {
8
9      /** Nombre completo del socio. */
10     private String nombre;
11
12     /**
13      * Registra un nuevo prestamo para este socio.
14      *
15      * @param material El material a prestar.
16      * @throws LimitePrestamosException si el socio ya tiene
17      *         el maximo
18      *         de prestamos activos.
19      */
20     public void registrarPrestamo(Material material)
21         throws LimitePrestamosException { ... }
22 }
```

- El Javadoc debe ser **real y útil**, no generado automáticamente con contenido vacío.
- Se verificará la generación del HTML de documentación con el comando `javadoc`.

4.2. Diagrama UML de Clases

Entregar un **diagrama UML de clases** completo que refleje:

- Todas las clases e interfaces del proyecto.
- Relaciones: herencia (**extends**), implementación (**implements**), asociación, composición/agregación.
- Visibilidad de atributos y métodos (+, -, #).

Herramienta recomendada: **draw.io**, **PlantUML**, **StarUML** o similar. Entregar dentro de la memoria.

4.3. Memoria Técnica

Redactar un documento PDF de **entre 10 y 20 páginas** que incluya:

1. **Portada:** nombre del alumno, curso, módulo, fecha y temática elegida.

2. **Descripción del sistema:** qué hace la aplicación y para quién está pensada.
3. **Análisis y diseño:** decisiones de diseño, justificación de la arquitectura de paquetes y de las colecciones elegidas.
4. **Manual de usuario:** capturas o listados de pantalla con explicación de cómo usar cada funcionalidad.
5. **Explicación técnica de los conceptos aplicados:** un apartado por cada requisito técnico (herencia, interfaces, streams, etc.) explicando *dónde y por qué* se usa en el proyecto.
6. **Dificultades encontradas y soluciones adoptadas.**
7. **Conclusiones:** qué has aprendido y qué mejorarías.
8. **Referencias:** documentación, tutoriales u otras fuentes consultadas.

5. Apartado de ampliación — Subir nota

Los siguientes apartados son **opcionales** y permiten subir la nota. No son requisito para aprobar. Requieren **investigación y aprendizaje autónomo** de contenidos no impartidos en clase.

Si no se llega a una nota mínima de 5 puntos en la parte obligatoria este apartado no contará, así como tampoco si hay algún RA suspenso.

Se valorará la profundidad y la integración real en el proyecto (no un añadido superficial).

5.1. Persistencia con Base de Datos — JDBC (*hasta +1,5 puntos*)

En lugar de (o además de) los ficheros CSV, persistir los datos usando una **base de datos relacional** mediante JDBC:

- Configurar una base de datos (se recomienda **SQLite** por simplicidad, o **MySQL**).
- Implementar un patrón **DAO** (Data Access Object) para cada entidad principal.
- CRUD completo en base de datos.
- Uso de **PreparedStatement** para todas las consultas (nunca concatenación de strings SQL).
- Gestión correcta de conexiones (**close** en **finally** o *try-with-resources*).

Documentar en la memoria cómo configurar y arrancar la base de datos.

5.2. Interfaz Gráfica con JavaFX (*hasta +1,5 puntos*)

Sustituir (o complementar) la interfaz de consola por una **interfaz gráfica con JavaFX**:

- Al menos **3 ventanas o vistas distintas** funcionales.

- Uso de controles JavaFX: `TableView`, `ListView`, `TextField`, `ComboBox`, `Button`, etc.
- Navegación entre vistas.
- Si se combina con FXML y el patrón MVC, se valorará adicionalmente.

La configuración del proyecto JavaFX debe estar documentada (`pom.xml` si Maven, o `module-info.java`).

5.3. Patrones de Diseño *(hasta +1 punto)*

Identificar y aplicar de forma justificada **al menos dos patrones de diseño**:

Patrón	Descripción orientativa
Singleton	Para la clase que gestiona la carga/guardado de datos
Factory Method	Para la creación de objetos polimórficos
Observer	Para notificaciones entre componentes
Strategy	Para distintos algoritmos de búsqueda u ordenación intercambiables
Decorator	Para añadir comportamiento dinámico a objetos

Documentar en la memoria qué patrón se usó, dónde, por qué y con un diagrama UML del patrón.

5.4. Serialización de Objetos *(hasta +0,5 puntos)*

- Las clases de dominio implementan `Serializable`.
- Guardar y cargar el estado completo de la aplicación desde fichero binario (`.dat`).
- Coexistir con la persistencia en CSV: ofrecer ambas opciones al usuario desde el menú.

5.5. Hilos y Concurrencia *(hasta +1 punto)*

- Ejemplos: carga de datos en background con barra de progreso en consola, simulación de procesos concurrentes, tareas programadas...
- Uso correcto de sincronización donde sea necesario (`synchronized`, `Lock`, etc.).
- Documentar el caso de uso en la memoria.

5.6. Tests Unitarios con JUnit 5 *(hasta +1 punto)*

- Mínimo **15 tests** que cubran casos normales, casos límite y casos de error.
- Uso de `@Test`, `@BeforeEach`, `@AfterEach`, `assertThrows()`, `assertEquals()`, etc.

- Los tests deben poder ejecutarse desde Maven/Gradle o desde el IDE sin configuración adicional.

6. Criterios de evaluación y rúbrica

Parte obligatoria (sobre 10 puntos)

Criterio	Peso	Indicadores de evaluación
Estructura y organización	0,5 pts	Paquetes lógicos, código limpio, convenciones de nombres
Diseño OOP	1 pt	Encapsulamiento, <code>toString/equals/hashCode</code> , coherencia
Herencia y polimorfismo	1 pt	Jerarquía con sentido, polimorfismo real, <code>@Override</code>
Clases abstractas e interfaces	1 pt	Uso justificado, interfaces como tipos
Colecciones y genéricos	1 pt	3 tipos distintos justificados, genérico propio
Excepciones personalizadas	0,5 pts	3 excepciones propias, <code>try-catch</code> correcto, mensajes útiles
Persistencia en ficheros	1 pt	Carga y guardado automáticos, CSV correcto, gestión de errores
Streams y lambdas	1 pt	5 usos reales y variados
Interfaz de consola	1 pt	CRUD completo, validaciones, mensajes claros
Javadoc	0,5 pts	Todas las clases/métodos públicos documentados
Diagrama UML	0,5 pts	Completo, correcto, con relaciones y cardinalidades
Memoria técnica	1 pt	Estructura completa, justificaciones técnicas, manual
Calidad general	0,5 pts	Cohesión del proyecto, creatividad, complejidad

Apartados de ampliación

Apartado	Puntos extra máximos
JDBC + DAO	+1,5
JavaFX	+1,5
Patrones de diseño	+1,0
Serialización	+0,5
Hilos y concurrencia	+1,0
JUnit 5	+1,0

Nota: La nota máxima no es 10. Los puntos de ampliación solo se suman si la parte obligatoria supera el **5**. Por ejemplo, si la parte obligatoria obtiene un 7 y los extras suman +2, la nota final será **9**.

7. Penalizaciones

Situación	Penalización
Entrega fuera de plazo	−5 pt
Código copiado de otro alumno (plagio)	0 para ambos
Código generado por IA sin comprensión	Hasta −3 pts (defensa oral)
Javadoc ausente o generado vacío automáticamente	−0,5 pts/comentario
Diagrama UML ausente	−5 pts
La aplicación no compila ni ejecuta	−10 pts
No persisten datos entre ejecuciones	−3 pt

Nota sobre el uso de IA: Se permite usar asistentes de IA (Copilot, ChatGPT, etc.) como herramienta de apoyo, igual que se permite buscar en StackOverflow. Lo que **no** está permitido es entregar código que no comprendes. Se podrá solicitar una **defensa oral** de cualquier parte en cualquier momento del proyecto.

8. Forma de entrega

Entregar **todo** a través de la plataforma del instituto (Moodle) en un **único archivo ZIP** con la siguiente estructura:

```
[Apellido_Nombre]_ProyectoFinal/
|-- src/                <- Código fuente completo
|-- doc/                <- HTML generado por javadoc
|-- recursos/          <- Ficheros CSV/TXT de datos de
    ejemplo
|-- UML_[Apellido].pdf  <- Diagrama UML
|-- Memoria_[Apellido].pdf <- Memoria técnica
'-- README.txt          <- Instrucciones para ejecutar
```

El README.txt debe indicar:

- IDE y versión de Java usados.
- Dependencias externas (si las hay) y cómo instalarlas.
- Cómo compilar y ejecutar.
- Credenciales de base de datos (si aplica el apartado de ampliación).

9. Sprints voluntarios

Fecha	Sprint
Viernes 8	Comunicar temática elegida al profesor
Viernes 15	Revisión voluntaria: entregar diagrama UML borrador para feedback
Viernes 22	Revisión voluntaria: demo del funcionamiento básico
Viernes 29 (entrega)	Entrega definitiva en Moodle

Las revisiones intermedias son voluntarias pero **muy recomendadas**: el profesor podrá orientarte y evitarás descubrir problemas de diseño en la última semana.

10. Consejos finales

1. **Empieza por el diseño, no por el código.** Dibuja el UML antes de escribir la primera línea.
2. **Divide el trabajo en semanas:** semana 1 → modelo y herencia; semana 2 → colecciones, excepciones y ficheros; semana 3 → streams, menú y pulido; semana 4 → documentación y ampliaciones.
3. **Usa Git desde el primer día.** Aunque no sea obligatorio, te salvará de perder trabajo.
4. **Prueba continuamente.** No esperes a tener todo para ejecutar. Prueba cada clase al crearla.
5. **El código que no entiendes no es tuyo.** Si usas IA o copias un fragmento, asegúrate de entenderlo línea a línea.